

KuchenApp

Projektdokumentation

Desktop-Anwendung zur Verwaltung von
Kuchen-Listen und IServ-Mail-Versand

Sprache: Python 3.10+
Framework: PySide6 (Qt 6)
Repository: github.com/Albazos/Kuchen
Stand: 17. Mai 2026

Inhaltsverzeichnis

1	Projektuebersicht	2
1.1	Funktionsumfang	2
1.2	Technologie-Stack	2
2	Projektstruktur	2
2.1	Verzeichnisstruktur	3
2.2	Trennung: develop vs. release	3
3	Architektur und Entwurfsmuster	4
3.1	Gesamtarchitektur	4
3.2	Verwendete Entwurfsmuster	4
3.2.1	Singleton-Pattern (AppLogger)	4
3.2.2	Observer-Pattern (Qt Signal/Slot)	5
3.2.3	Model-View-Pattern (Qt MVC)	5
4	Module im Detail	5
4.1	KuchenMain.py – Hauptfenster	5
4.2	DataManager.py – Datenverwaltung	6
4.3	SettingsManager.py – Einstellungsverwaltung	7
4.4	AppLogger.py – Singleton-Logger	8
4.5	KuchenMailManager.py – Mail-Listen-Dialog	8
4.6	LoginDialog.py – IServ-Login	8
4.7	IServMailManager.py – Mail-Versand	8
4.8	IServAPI.py – IServ-API	8
5	Datenpersistenz	9
5.1	CSV-Format	9
5.2	Settings (INI)	9
6	Build-System und CI/CD	9
6.1	build_release.py – Lokaler Build	9
6.2	PyInstaller – Standalone-Builds	10
6.3	GitHub Actions – CI/CD-Pipeline	10
7	Sicherheitsaspekte	11
8	Benutzerhandbuch	11
8.1	Installation (Source-Version)	11
8.2	Installation (Standalone)	12
8.3	Bedienung	12
9	Glossar	12

1 Projektuebersicht

KuchenApp ist eine Desktop-Anwendung, die im Rahmen eines Schulprojekts (Berufsschule, Fachinformatiker Anwendungsentwicklung) entwickelt wurde. Die Anwendung dient zur Verwaltung einer Kuchen-Liste fuer eine Schulklasse und ermoeeglicht den automatisierten Versand von Erinnerungs-E-Mails ueber das IServ-System.

1.1 Funktionsumfang

- CSV-Dateien importieren, bearbeiten und speichern
- Dynamische Tabellendarstellung mit Sortier- und Suchfunktion
- Aufsteigende und absteigende Sortierung per ComboBox-Auswahl
- Verwaltung einer Empfaenger-Mailliste (separater Dialog)
- Automatische HTML-Tabellengenerierung fuer E-Mail-Inhalte
- E-Mail-Versand ueber IServ-SMTP mit editierbarem Subject
- INI-basierte Einstellungsverwaltung
- Zentrales Dark-Blue-Theme ueber externe QSS-Datei
- Standalone-Build fuer Linux und Windows (PyInstaller)
- CI/CD ueber GitHub Actions mit automatischem Release

1.2 Technologie-Stack

Komponente	Technologie
Programmiersprache	Python 3.10+
GUI-Framework	PySide6 (Qt 6 for Python)
UI-Design	Qt Designer (.ui-Dateien)
Datenpersistenz	CSV (comma-separated values)
Einstellungen	INI-Dateien (configparser)
E-Mail-Versand	smtpplib (SMTP_SSL)
Build / Packaging	PyInstaller
CI/CD	GitHub Actions
Versionsverwaltung	Git (eigenes Repository)

Tabelle 1: Verwendete Technologien

2 Projektstruktur

Das Projekt ist ein **eigenstaendiges Git-Repository** mit klarer Trennung zwischen Entwicklungs- und Release-Code.

2.1 Verzeichnisstruktur

Gesamtstruktur

```

Kuchen/
|-- build_release.py          # Build-Skript (Source +
    Download)
|-- requirements.txt          # Python-Abhaengigkeiten
|-- .gitignore
|-- .github/workflows/        # CI/CD Pipeline
|   +-- build_release.yml
|-- develop/                  # Entwicklungsumgebung
|   |-- KuchenMain.py         # Hauptprogramm (Entry Point)
|   |-- src/                  # Quellcode-Module
|       |-- __init__.py
|       |-- AppLogger.py
|       |-- DataManager.py
|       |-- IServAPI.py
|       |-- IservMailManager.py
|       |-- KuchenMailManager.py
|       |-- LoginDialog.py
|       +-- SettingsManager.py
|   |-- ui/                   # UI-Dateien
|       |-- __init__.py
|       |-- UIMainWindow.ui / _ui.py
|       |-- UIMailsDialog.ui / _ui.py
|       +-- UILoginDialog.ui / _ui.py
|   |-- assets/
|       |-- kuchen_icon.svg
|       |-- chevron_down.svg
|       +-- theme_dark_blue.qss
|   +-- Data/
|       |-- StandardCakeData.csv
|       |-- Settings/settings.example.ini
|       +-- Klassen/StandardKlasse.csv
+-- release/                  # Generierte Releases (
    gitignored)
    |-- KuchenApp/            # Source-Bundle
    |-- KuchenApp.zip
    |-- KuchenApp_standalone_linux.zip
    +-- KuchenApp_standalone_windows.zip

```

2.2 Trennung: develop vs. release

develop/

Enthaelt den vollstaendigen Quellcode, .ui-Designdateien sowie neutrale Template-Dateien. Hier wird aktiv entwickelt.

release/

Wird durch `build_release.py` automatisch generiert und via `.gitignore` nicht versioniert. Enthält nur die für den Endbenutzer nötigen Dateien. Die CSV-Dateien werden mit leeren Standard-Headern ausgeliefert.

Hinweis zu Standard- und lokalen Daten

Im Repository sollten nur neutrale Template-Dateien versioniert werden. Reale Nutzdaten aus dem schulischen Alltag können lokal weiterverwendet werden, sollten aber für ein öffentliches Repository nicht eingchecked werden.

3 Architektur und Entwurfsmuster

3.1 Gesamtarchitektur

Die Anwendung folgt einer **Schichtenarchitektur**:

1. **Präsentationsschicht** – GUI-Klassen (`CMainWindow`, `CKuchenDialog`, `CLoginDialog`), Qt-UI-Dateien
2. **Logikschicht** – Datenverarbeitung (`DataManager`), Einstellungen (`SettingsManager`), Mail-Versand (`IServMailManager`)
3. **Datenzugriffsschicht** – CSV-Dateien, INI-Konfiguration, `IServ-SMTP-API`

3.2 Verwendete Entwurfsmuster

3.2.1 Singleton-Pattern (AppLogger)

Der `AppLogger` implementiert das **Singleton-Pattern** über Pythons `__new__`-Methode. Damit existiert während der gesamten Laufzeit exakt eine `Logger`-Instanz.

Fachliche Begründung

Alle Klassen sollen Fehlermeldungen an die GUI weiterleiten können, ohne eine Referenz auf das Hauptfenster zu benötigen. Der Singleton stellt sicher, dass alle `AppLogger()`-Aufrufe dieselbe Instanz zurückgeben und somit die gleichen Qt-Signale nutzen.

```
1 class AppLogger(QObject):
2     errorOccurred = Signal(str, str)
3     warningOccurred = Signal(str, str)
4     infoOccurred = Signal(str, str)
5
6     _instance = None
7
8     def __new__(cls):
9         if cls._instance is None:
10             cls._instance = super().__new__(cls)
11         return cls._instance
12
```

```

13     def __init__(self):
14         if not hasattr(self, '_initialized'):
15             super().__init__()
16             self._initialized = True

```

Listing 1: Singleton-Implementierung

3.2.2 Observer-Pattern (Qt Signal/Slot)

Die Kommunikation zwischen den Schichten erfolgt ueber das **Observer-Pattern**, implementiert durch Qt's Signal/Slot-Mechanismus.

- AppLogger emittiert Signale (errorOccurred, warningOccurred, infoOccurred)
- CMainWindow verbindet diese mit Slot-Methoden, die QMessageBox-Dialoge anzeigen
- QStandardItemModel.dataChanged benachrichtigt bei Zellenaenderungen in der Tabelle

Fachliche Begrueundung

Lose Kopplung – der DataManager und IservMailManager kennen die GUI nicht, sondern kommunizieren ueber den AppLogger. Dies ermoeoglicht eine saubere Trennung der Schichten.

3.2.3 Model-View-Pattern (Qt MVC)

Die Tabellendarstellung nutzt Qt's **Model-View**-Architektur:

Rolle	Implementierung
Model	QStandardItemModel – haelt die Tabelleneintraege
View	QTableView – zeigt die Daten an
Controller	Window-Klasse vermittelt zwischen DataManager und Model

4 Module im Detail

4.1 KuchenMain.py – Hauptfenster

CMainWindow erbt von QMainWindow und Ui_MainWindow (Multiple Inheritance, empfohlener Qt-Ansatz). Es ist der **Entry Point** der Anwendung.

_createRadioButtons()

Befuellst die Sortier-ComboBox dynamisch mit allen aktuellen CSV-Headern.

SortColumn()

Liest die ausgewaehlte Spalte aus cmbSortField sowie die Richtung aus cmbSortDirection und ruft DataManager.sortData() entsprechend auf.

Search(aText)

Filtert die Originaldaten (mData) nach dem Suchbegriff. Ist in der Sortier-ComboBox

eine Spalte ausgewählt, wird nur in der entsprechenden Spalte gesucht. Ansonsten: Volltextsuche ueber alle Spalten. Nach dem Filtern wird die Sortierung erneut angewandt.

CellEdited(aItem)

Aktualisiert den bearbeiteten Tabellenwert in `mSortedData`. Da die gefilterten und sortierten Ansichten auf dieselben Zeilenobjekte wie die Originaldaten verweisen, bleibt `mData` dabei konsistent.

FillTable()

Loescht das `QStandardItemModel`, setzt Header und fuehlt die Tabelle aus `mSortedData`. Aktualisiert zusaetzlich die Metaanzeige mit der Anzahl der Eintraege.

SendMails()

Zeigt einen Bestaetigungsdialog mit der Empfaengeranzahl, oeffnet den Login-Dialog und wertet den Tri-State-Rueckgabewert aus (`None`=abgebrochen, `True`=gesendet, `False`=fehlgeschlagen).

_load_app_theme()

Laedt die zentrale Datei `theme_dark_blue.qss`, ersetzt den Platzhalter fuer den Asset-Pfad und bindet das Theme global in die `QApplication` ein.

ImportFile()

Oeffnet einen `QFileDialog`, importiert die CSV und aktualisiert die Sortier-ComboBox dynamisch anhand der neuen Header.

PyInstaller-Kompatibilitaet

Im `__main__`-Block wird `sys.frozen` geprueft, um den Icon-Pfad korrekt aufzuloesen (`sys._MEIPASS` im Bundle vs. `__file__` im Entwicklungsmodus).

4.2 DataManager.py – Datenverwaltung

Der `DataManager` ist die zentrale Klasse fuer alle Datenoperationen. Er verwaltet zwei unabhaengige Datensaeetze:

Datensatz	Attribute
Haupttabelle (Kuchen)	<code>mData</code> , <code>mSortedData</code> , <code>mMainHeaders</code>
Mail-Tabelle (Empfaenger)	<code>mMailData</code> , <code>mSortedMailData</code> , <code>mMailHeaders</code>

Duale Datenhaltung

`mData` enthaelt immer die *unsortierten* Originaldaten. `mSortedData` ist eine gefilterte/sortierte Kopie, die in der Tabelle angezeigt wird. Dies ermoeeglicht das Zuruecksetzen von Sortierung und Suche.

Schluesselfunktionen:

LoadStandardFile(aForMainTable)

Laedt die CSV-Datei, deren Name in den Settings steht. Verwendet `csv.Sniffer` zur

automatischen Dialekt-Erkennung (Trennzeichen, Quoting). Extrahiert die erste Zeile als Header und merkt sich das erkannte Trennzeichen fuer spaeteres Speichern.

ImportFile(aPath)

Importiert eine externe CSV. Speichert den Dateinamen getrennt fuer Haupt- und Maildaten, damit beim naechsten `SaveFile()` der passende Name in die Settings uebernommen wird.

SaveFile(aForMainTable)

Speichert die Daten als CSV. Wurde zuvor eine Datei importiert, aktualisiert `SaveFile` automatisch die Settings mit dem neuen Dateinamen und behaelt das urspruenglich erkannte Trennzeichen bei.

sortData(aSortColumnIndex, aReverse, aForMain)

Sortiert `mSortedData` nach der gegebenen Spalte. Der Parameter `aReverse` steuert die Richtung.

_padRows(aData, aHeaderLen)

Fuellt Datenzeilen mit leeren Strings auf, falls sie weniger Spalten als der Header haben – Robustheit beim Import unvollstaendiger CSVs.

createCompleteMailData()

Bereitet Mail-Daten auf: extrahiert E-Mail-Adressen und generiert einen HTML-String der Kuchen-Tabelle fuer den E-Mail-Body.

createHtmlDataString()

Erzeugt eine formatierte HTML-Tabelle mit Inline-CSS. Verwendet `html.escape()` zur Vermeidung von XSS-Schwachstellen.

4.3 SettingsManager.py – Einstellungsverwaltung

Verwaltet die Anwendungseinstellungen in einer INI-Datei (`Data/Settings/settings.ini`) ueber Pythons `configparser`-Modul.

```
1  [Klassen]
2  Dateiname = StandardKlasse.csv
3
4  [CakeData]
5  Dateiname = StandardCakeData.csv
6
7  [IServ]
8  Domain = example.org
```

Listing 2: settings.ini – Konfigurationsstruktur

Default-Werte

Die Klasse definiert ein `_DEFAULTS`-Dictionary. Beim Instanzieren werden zuerst die Defaults geladen, dann die INI-Datei daruebergelegt. So sind immer Fallback-Werte vorhanden.

4.4 AppLogger.py – Singleton-Logger

Implementiert das Singleton-Pattern (siehe Abschnitt 3.2.1). Stellt drei Schweregrade bereit:

Methode	Dialog	Verwendung
<code>error()</code>	<code>QMessageBox.critical</code>	Kritische Fehler
<code>warning()</code>	<code>QMessageBox.warning</code>	Warnungen
<code>info()</code>	<code>QMessageBox.information</code>	Informationen

Jeder Aufruf emittiert ein Qt-Signal mit Titel und Nachricht. Die GUI verbindet diese Signale mit den entsprechenden `QMessageBox`-Methoden.

4.5 KuchenMailManager.py – Mail-Listen-Dialog

`CKuchenDialog` ist ein modaler Dialog (`QDialog`) zur Verwaltung der Empfaenger-Mailliste. Er teilt sich den `DataManager` mit dem Hauptfenster und arbeitet auf den Mail-Daten (`mMailData`).

Die Funktionalitaet ist analog zum Hauptfenster: Sortierung per `ComboBox`, Suche, Import, Speichern, Zeilen hinzufuegen/loeschen und zentrale Theme-Nutzung. Der Button `pbnAddColumn` ist bereits in der Oberflaeche vorhanden, loggt derzeit aber nur, dass die Funktion noch nicht implementiert ist.

4.6 LoginDialog.py – IServ-Login

`CLoginDialog` ist ein modaler Dialog fuer die SMTP-Authentifizierung. Der Dialog zeigt zusaetzlich die Empfaengeranzahl und ein editierbares Subject-Feld an.

Tri-State-Logik

- `mSendState = None` – Benutzer hat abgebrochen
- `mSendState = True` – Mails erfolgreich gesendet
- `mSendState = False` – Fehler beim Senden (Dialog bleibt offen)

4.7 IServMailManager.py – Mail-Versand

Erstellt eine Verbindung zur IServ-API und sendet E-Mails ueber `SMTP_SSL`. Das Passwort wird im `finally`-Block auf `None` gesetzt, um es nicht unnoetig im Speicher zu halten.

Der Mail-Subject wird nicht mehr fest verdrahtet, sondern aus dem Login-Dialog uebernommen. Ist das Subject-Feld leer, wird auf "Kuchen reminder" zurueckgefallen.

Fehlerbehandlung faengt spezifische Exceptions: `OSError`, `smtplib.SMTPException`, `ValueError`, `ConnectionError`.

4.8 IServAPI.py – IServ-API

Eine angepasste Version der IServ-API, die **ausschliesslich** die Python-Standardbibliothek verwendet (kein `pip install` noetig).

Externe Abhaengigkeit	Stdlib-Ersatz
requests	urllib.request
beautifulsoup4	Eigene _HTMLNode-Klasse
SMTP-Verbindung	smtpplib.SMTP_SSL
MIME-Nachrichten	email.mime

5 Datenpersistenz

5.1 CSV-Format

Die Anwendung speichert Daten als CSV-Dateien (Comma-Separated Values). Beim Laden wird `csv.Sniffer` zur automatischen Erkennung des Trennzeichens verwendet – so werden sowohl Komma- als auch Semikolon-getrennte Dateien korrekt gelesen.

```
1 Name , CakeCount , Hanuta , Waffel , Date
```

Listing 3: StandardCakeData.csv – Header-Vorlage

Dynamische Header

Die Header werden nicht hartcodiert, sondern aus der ersten Zeile der CSV-Datei gelesen. Die GUI befüllt ihre Sortier-ComboBoxen dynamisch aus diesen Headern.

5.2 Settings (INI)

Einstellungen werden ueber Pythons `configparser`-Modul in einer INI-Datei verwaltet. Die Datei wird mit Defaults initialisiert und bei Aenderungen (z. B. Import einer neuen CSV) automatisch aktualisiert.

6 Build-System und CI/CD

Das Build-System ist zweigeteilt: **lokal** wird nur das Source-Bundle erstellt, **Standalone-Builds** fuer Linux und Windows laufen ausschliesslich ueber GitHub Actions.

6.1 build_release.py – Lokaler Build

Verwendung

```
python build_release.py          # Release-Ablauf mit
    Versionssprung
python build_release.py --source  # Nur Source-Bundle bauen
python build_release.py --standalone # Standalone fuer
    aktuelles OS bauen
```

Ablauf bei normalem Aufruf ohne Flags:

1. **Version:** Patch-Version in `version.ini` wird erhoeht
2. **Clean:** Alter `release/`-Ordner wird geloescht
3. **UI-Kompilierung:** `.ui` → `_ui.py` via `pyside6-uic`
4. **Kopieren:** Quelldateien in die Release-Struktur
5. **README:** Automatische `README.txt`-Generierung
6. **ZIP:** Source-Bundle als `KuchenApp.zip`
7. **Release:** Git- und GitHub-Release-Ablauf mit Upload der Artefakte

Voraussetzung fuer Download

Die GitHub CLI (`gh`) muss installiert und authentifiziert sein:
`gh auth login`

6.2 PyInstaller – Standalone-Builds

PyInstaller verpackt Python-Interpreter, PySide6 und alle Quelldateien in eine einzelne ausfuehrbare Datei. Die Standalone-Builds werden **nur in der CI-Pipeline** ausgefuehrt.

Option	Bedeutung
<code>-onefile</code>	Einzelne Datei statt Ordner
<code>-windowed</code>	Kein Konsolenfenster (GUI-App)
<code>-add-data</code>	Icon und Data-Verzeichnis einbetten

Laufzeit-Pfadaufloesung: Da PyInstaller Dateien in ein temporaeres Verzeichnis (`sys._MEIPASS`) entpackt, aber `Data/` beschreibbar sein muss, wird `Data/` neben die Executable kopiert:

```

1 def _get_base_dir():
2     if getattr(sys, 'frozen', False):
3         return os.path.dirname(sys.executable)
4     return os.path.dirname(os.path.dirname(
5         os.path.abspath(__file__)))

```

Listing 4: Pfadaufloesung fuer PyInstaller

6.3 GitHub Actions – CI/CD-Pipeline

Der Workflow `.github/workflows/build_release.yml` baut automatisch fuer beide Betriebssysteme.

Pipeline-Uebersicht

Trigger: Version-Tag (`v*`)
Jobs: Matrix-Build fuer Linux und Windows
Release: Haengt die Standalone-ZIPs an das GitHub Release an

Workflow-Ablauf:

1. **Tag-Push** – der lokale Release-Build erstellt und pusht einen Version-Tag (v*).
2. **Matrix-Build** – GitHub Actions baut je eine Standalone-ZIP fuer Linux und Windows mit `build_release.py --standalone`.
3. **Release-Upload** – der Workflow haengt die Standalone-ZIPs an das GitHub Release an.
4. **Source-Upload** – das lokale Skript laedt anschliessend `KuchenApp.zip` zum gleichen Release hoch.

Release ausloesen

```
python build_release.py
```

Nur Source-Bundle in der CI oder lokal

```
python build_release.py --source
```

7 Sicherheitsaspekte

Massnahme	Beschreibung
HTML-Escaping	Alle Tabelleninhalte werden mit <code>html.escape()</code> in der Mail-HTML-Generierung escaped (XSS-Schutz)
Passwort-Loeschung	SMTP-Passwort wird im <code>finally</code> -Block auf <code>None</code> gesetzt
Spezifische Exceptions	Nur erwartete Exception-Typen werden gefangen (<code>OSError</code> , <code>csv.Error</code> , <code>smtplib.SMTPException</code>)
SMTP_SSL	Verbindung zu IServ ueber SSL-verschluesseltes SMTP (Port 465)
.gitignore	Release-Artefakte, <code>__pycache__</code> , virtuelle Environments werden nicht versioniert

Tabelle 2: Sicherheitsmassnahmen

8 Benutzerhandbuch

8.1 Installation (Source-Version)

Schnellstart

```
pip install -r requirements.txt
python develop/KuchenMain.py
```

8.2 Installation (Standalone)

Die Standalone-Version benoetigt keine Installation:

OS	Starten
Linux	<code>./KuchenApp</code>
Windows	Doppelklick auf <code>KuchenApp.exe</code>

8.3 Bedienung

1. **Daten laden:** Beim Start wird die zuletzt verwendete CSV automatisch geladen.
2. **Import:** Ueber den *Import*-Button kann eine andere CSV-Datei geladen werden. Spalten und Sortier-ComboBox passen sich automatisch an.
3. **Bearbeiten:** Zellen koennen direkt in der Tabelle bearbeitet werden (Doppelklick).
4. **Zeilen hinzufuegen/loeschen:** Ueber die Buttons *+ New entry* und *Delete*.
5. **Sortierung:** ComboBoxes in der Sidebar. *Ascending/Descending* waehlt die Richtung.
6. **Suche:** Das Suchfeld *Search* filtert live. Bei aktiver Sortier-Spalte wird nur in der entsprechenden Spalte gesucht.
7. **Speichern:** Ueber den *Save*-Button. Bei importierten Dateien wird der neue Dateiname in den Settings gespeichert.
8. **Mail-Liste:** Ueber *Open mail list* oeffnet sich ein separater Dialog zur Verwaltung der Empfaenger.
9. **Mails senden:** Ueber *Send mails*. Es erscheint eine Bestaetigung mit der Empfaengeranzahl, dann der Login-Dialog mit editierbarem Subject.

9 Glossar

CSV

Comma-Separated Values – Tabellenformat mit Trennzeichen

INI

Initialisierungsdatei – Einfaches Konfigurationsformat

PySide6

Offizielle Python-Bindings fuer das Qt 6 Framework

Qt Designer

Visueller Editor fuer Qt-Oberflaechen (.ui-Dateien)

PyInstaller

Tool zum Verpacken von Python-Programmen als Standalone

SMTP_SSL

Simple Mail Transfer Protocol ueber SSL-Verschluesselung

Singleton

Entwurfsmuster: Garantiert genau eine Instanz einer Klasse

Observer

Entwurfsmuster: Objekte werden ueber Zustandsaenderungen benachrichtigt (in Qt: Signal/Slot)

MVC

Model-View-Controller – Architekturmuster zur Trennung von Daten, Darstellung und Steuerung

CI/CD

Continuous Integration / Continuous Deployment – Automatisierte Build- und Release-Pipelines

XSS

Cross-Site Scripting – Sicherheitsluecke durch unescaped HTML-Input

IServ

Schulplattform mit E-Mail, Kalender und Dateiverwaltung

GitHub Actions

CI/CD-Dienst von GitHub fuer automatisierte Workflows